

CM2005 OOP Endterm Assignment 2025 - DJ Studio (OtoDecks Enhanced)

Student Name: Devon Maya Xavior
UOL Student ID: 200618744
SIM Student ID: 10204322
Email: dmxavior001@mymail.sim.edu.sg

R1: DJ STUDIO BASIC FUNCTIONALITY

R1A: Load audio files into audio players

DJ Studio utilises the audio file chooser (via JUCE's *FileBrowserComponent* and *FileDragAndDropTarget* support) to let users select audio files. JUCE's *AudioTransportSource* and *AudioFormatManager* classes will then read the audio data. Both the "Select File" and "Add to Deck 1/2" buttons will trigger the `loadFile` method in `DJAudioPlayer.cpp` to create an `AudioFormatReaderSource` from the file and then set it on the transport source.

```
if (button == &loadButton)
{
    auto fileChooserFlags =
    juce::FileBrowserComponent::canSelectFiles;
    fChooser.launchAsync(fileChooserFlags, [this](const juce::FileChooser& chooser)
    {
        player->loadFile(juce::URL{chooser.getResult()});
        // and now the waveformDisplay as well
        waveformDisplay.loadURL(juce::URL{chooser.getResult()});

        double length = player->getLengthInSeconds();
        waveformDisplay.setTrackLength(length);
    });
}

// File Drag and Drop Events
bool DeckGUI::isInterestedInFileDrag(const juce::StringArray &files)
{
    DBG("DeckGUI::isInterestedInFileDrag");
    return true;
}

void DeckGUI::filesDropped(const juce::StringArray &files, int x, int y) // ** Own Codes **
{
    DBG("DeckGUI::filesDropped");
    if (files.size() == 1)
    {
        juce::File droppedFile(files[0]);
        juce::URL fileURL(droppedFile);
        player->loadFile(fileURL);
        waveformDisplay.loadURL(fileURL);
    }
}

// Audio Playback Controls ** Own Codes **
void DJAudioPlayer::loadFile(juce::URL audioURL)
{
    auto* reader = formatManager.createReaderFor(audioURL.createInputStream(false)); ⚠️ 'createInputStream' is depr
    if (reader != nullptr)
    {
        std::unique_ptr<juce::AudioFormatReaderSource> newSource (new juce::AudioFormatReaderSource (reader,
true));
        transportSource.setSource (newSource.get(), 0, nullptr, reader->sampleRate);
        readerSource.reset (newSource.release());
    }
}
```

R1B: Play two tracks simultaneously

The player comes with 2 decks that can play their tracks concurrently. The decks are controlled under the DeckGUI class that is associated with an instance of DJAudioPlayer, which work together to allow playback of 2 tracks at the same time. The main GUI layout in DJ Studio lets users load the tracks into each deck (either by pressing “Select File” or “Add to Deck 1 or 2” buttons). The respective DeckGUI component then communicates with DJAudioPlayer to control the playback of each track independently.

```
// Audio Processing Methods
void MainComponent::prepareToPlay (int samplesPerBlockExpected, double sampleRate)
{
    // Audio Players
    player1.prepareToPlay(samplesPerBlockExpected, sampleRate);
    player2.prepareToPlay(samplesPerBlockExpected, sampleRate);

    // Mixer Source
    mixerSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
    mixerSource.addInputSource(&player1, false);
    mixerSource.addInputSource(&player2, false);
}

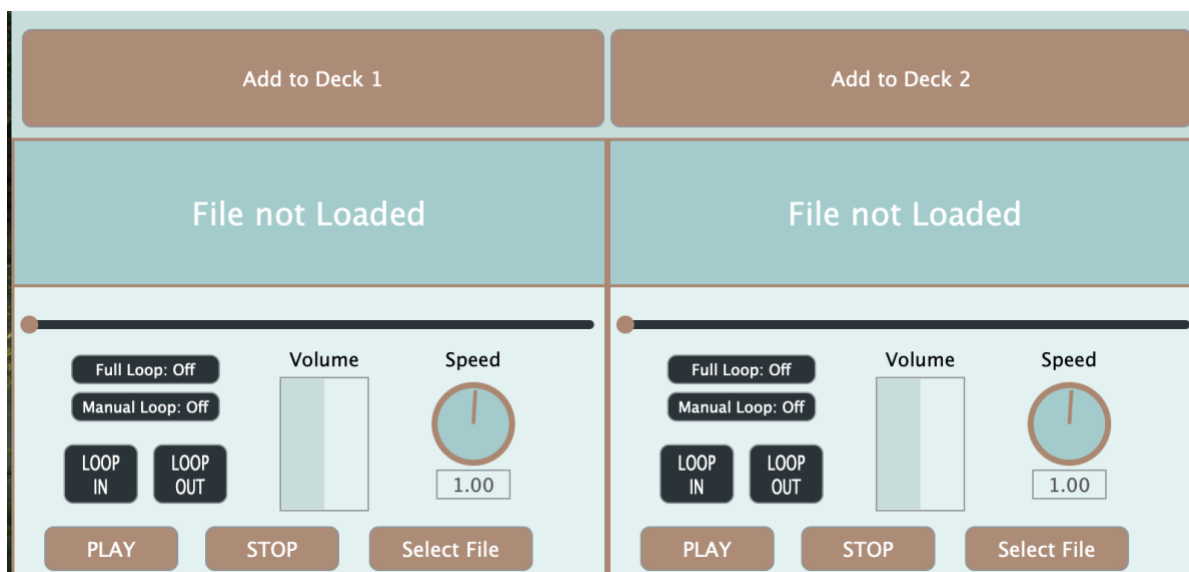
void MainComponent::getNextAudioBlock(const juce::AudioSourceChannelInfo& bufferToFill)
{
    // Pass audio to mixer source
    mixerSource.getNextAudioBlock(bufferToFill);
}

void MainComponent::releaseResources()
{
    // Release resources for players and mixer
    player1.releaseResources();
    player2.releaseResources();
    mixerSource.releaseResources();
}

void MainComponent::resized()
{
    // Layout playlist component in top half of component
    playlistComponent.setBounds(0, 0, getWidth(), getHeight() / 2);

    // Layout deck GUIs in the bottom half.
    int halfWidth = getWidth() / 2;
    int halfHeight = getHeight() / 2;

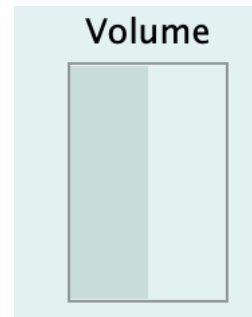
    deckGUI1.setBounds(0, halfHeight, halfWidth, halfHeight);
    deckGUI2.setBounds(halfWidth, halfHeight, halfWidth, halfHeight);
}
```



R1C: Mix tracks by varying volumes

Each deck (i.e., Deck 1 or 2) comes with a volume slider that adjusts the gain on corresponding DJAudioPlayer, allowing users to mix tracks by increasing/decreasing volumes independently. The *setGain* method in DJAudioPlayer controls the track volume. A Linear Bar slider is created in the DeckGUI component for this purpose.

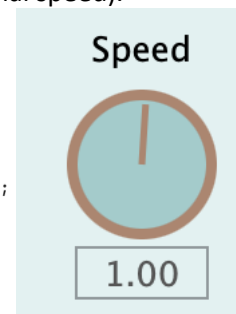
```
void DJAudioPlayer::setGain(double gain)
{
    if (gain < 0.0 || gain > 1.0)
    {
        DBG("DJAudioPlayer::setGain gain should be between 0 and 1");
    }
    else {
        transportSource.setGain(gain);
    }
}
```



R1D: Speed up/Slow down tracks

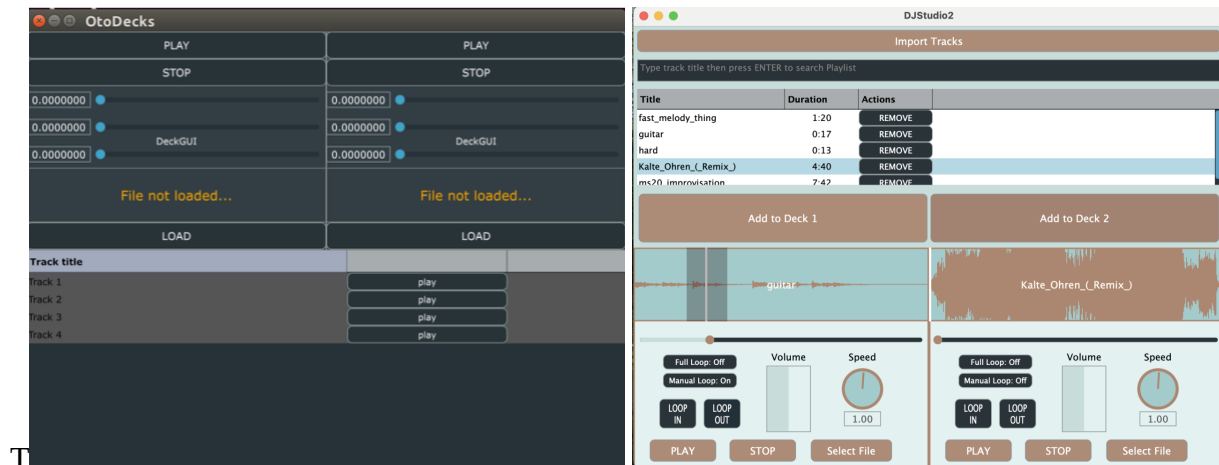
Playback speed is controlled by a rotary knob (speedSlider) in the DeckGUI. Adjusting slider changes the resampling ratio on the audio source, resulting in speeding up or slowing down of the track without affecting the track pitch significantly. The *setSpeed* method in DJAudioPlayer modifies the resampling ratio. A textbox below the knob shows the speed times (i.e., 0.5x to 3x; 1 being original speed).

```
void DJAudioPlayer::setSpeed(double ratio)
{
    if (ratio <= 0 || ratio > 100.0)
    {
        DBG("DJAudioPlayer::setSpeed ratio should be between 0 and 100");
    }
    else {
        resampleSource.setResamplingRatio(ratio);
    }
}
```



R2: CUSTOMISE USER INTERFACE

R2A: Different GUI Layout from original basic DeckGUI



Significant changes have been made to change the GUI Layout of the DJ player.

	Original Basic (OtoDecks)	New (DJ Studio)
Colour Scheme	Dark grey shades for background and main buttons; Bright orange and white for text. A generally “nightmode” or Dark scheme for the player.	Soft Teal shades for background and sliders, with Warm Sand accents for buttons and outlines. Colour scheme is softer and more of a “daymode” or Day scheme in terms of colours.
Layout	The top section of the interface displays the playback controls prominently. Just beneath these, a row of sliders for volume, speed, and track position is aligned with the waveform display and accompanying load buttons. The lower half of the player is entirely dedicated to the playlist component.	The interface is divided into two distinct sections. The upper half is dedicated to music library management, featuring the import button, track search bar, playlist display, and Add-to-Deck buttons. The lower half is reserved for playback control, displaying the waveform at the top, followed by an array of controls for playback, looping, volume, speed, and track position adjustments.
Buttons/Sliders (Controls)	Simple linear slider controls for volume, speed and position.	Volume control uses horizontal linear bar with soft teal fill to show volume levels. Speed is controlled by a rotary knob with textbox showing speed times. Position slider is kept as simple slider as it is most visually intuitive with marker position.
Extras	Player features only basic playback (play and stop) controls with playlist and track loading functions.	On top of basic features like OtoDecks, DJ Studio introduces Looping and Track search in playlist.

R2B: New Event Listeners (Loop Control)

In addition to the basic event listeners, extra listeners were implemented to manage loop control. A dedicated toggle enables full-track looping, allowing users to switch looping on or off. When manual loop mode is activated, the full-track loop is automatically disabled. In manual loop mode, separate Loop In and Loop Out buttons let users mark the start and end points of a custom loop section—these controls remain inactive unless manual loop mode is enabled.

```

// ** Own Codes **
fullLoopToggleButton.addListener(this);
manualLoopToggleButton.addListener(this);
loopInButton.addListener(this);
loopOutButton.addListener(this);

if (button == &loopInButton)
{
    if (manualLoopMode)
    {
        double pos = player->getCurrentPosition();
        loopStart = pos;
        DBG("Loop In set at: " << loopStart << " seconds");
        waveformDisplay.setLoopRegion(loopStart, loopEnd);
    }
    else
    {
        DBG("Manual loop mode is off. Enable it to set loop points.");
    }
}
if (button == &loopOutButton)
{
    if (manualLoopMode)
    {
        double pos = player->getCurrentPosition();
        loopEnd = pos;
        if (loopEnd > loopStart)
        {
            player->setManualLoop(loopStart, loopEnd);
            DBG("Loop Out set at: " << loopEnd << ". Manual loop active.");
            waveformDisplay.setLoopRegion(loopStart, loopEnd);
        }
        else
        {
            DBG("Invalid manual loop: Loop Out must be after Loop In.");
        }
    }
    else
    {
        DBG("Manual loop mode is off. Enable it to set loop points.");
    }
}
if (button == &fullLoopToggleButton)
{
    // Toggle full-track loop mode.
    fullLoopOn = !fullLoopOn;
    player->setLooping(fullLoopOn); // This method loops the entire track.
    fullLoopToggleButton.setText(fullLoopOn ? "Full Loop: On" : "Full Loop: Off");

    // When full-loop is enabled, disable manual loop mode.
    if (fullLoopOn)
    {
        manualLoopMode = false;
        manualLoopToggleButton.setText("Manual Loop: Off");
    }
}
if (button == &manualLoopToggleButton)
{
    manualLoopMode = !manualLoopMode;
    manualLoopToggleButton.setText(manualLoopMode ? "Manual Loop: On" : "Manual Loop: Off");

    if (!manualLoopMode)
    {
        // When turning off manual loop mode, clear the manual loop in the player.
        player->clearManualLoop();
        waveformDisplay.clearLoopRegion();
    }
    else
    {
        // When manual loop is enabled, disable full-track looping if it's active.
        fullLoopOn = false;
        fullLoopToggleButton.setText("Full Loop: Off");
        // Optionally, clear any previously set manual loop.
        player->clearManualLoop();
    }
}
}

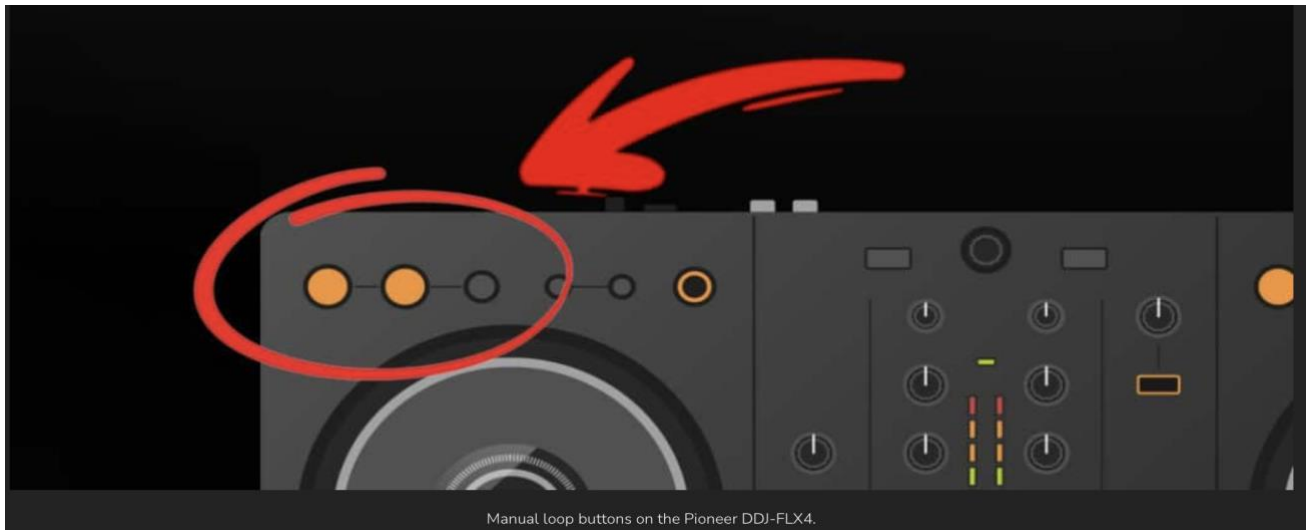
```

R3: ONE NEW FEATURE – PLAYLIST & LOOPING / MANUAL LOOPING

After researching DJ software such as Serato DJ and Rekordbox (from Crossfader), Looping was identified as a powerful and common feature on DJ applications, especially Manual Looping.

Manual Looping allows DJs to set a custom loop by marking a loop in (start) and a loop out (end) point, allowing DJs to get creative with their performing techniques.

On top of this, a simple playlist component is added to facilitate quick adding of tracks to the separate decks.



The Manual looping method in the aforementioned DJ software allows saving and layering (as you can see from above pictures). However, for the purpose of this assignment, I decided to start from the very basic, which is to just incorporate a Full track loop, allowing the track to play non-stop, and also selecting loop sections. Going forward, I may introduce a saving feature that allows pasting of selected loop sections into new tracks; and also Preset Loop buttons that allows 4-second, 8-second loops etc., very quickly with just a click of the button.

Playlist & Loop Features:

- **Full Track Loop:** Loops the whole track until loop is toggled off.
- **Manual Loop:** Once toggled on, user sets Loop In and Loop Out points (using respective buttons) to active loop section of the track. Once manual loop is toggled off, song will continue playing as per normal.
- **Playlist:** Users can import multiple tracks into the playlists, which allows quick add on, search and removal of tracks from the list.
- **Waveform Feedback:** Waveform display is updated to reflect selected loop region. Loop section is rendered with a shaded colour overlay to distinguish loop section from rest of track.



Constructs and Classes Used:

- **DJAudioPlayer:** Extended with methods like *setLooping*, *setManualLoop*, *clearManualLoop* and *checkLooping*
- **PlaylistComponent:** This class is responsible for managing and displaying the user's music library. It maintains a collection of tracks (stored as a vector of Track objects) and provides functionalities for:
 - o **Importing Tracks:** Add new tracks via file chooser.
 - o **Search:** Search bar allows filtering of track list based on track title.
 - o **Playlist Persistence:** The library persists between sessions by saving and loading track information (such as file path, title and duration) to/from a CSV file.
 - o **Presentation:** Updates a table view (using JUCE's TableListBoxModel) to present track list.
- **Track Class:** Encapsulates the data for an individual audio track.
 - o **Data encapsulation:** stores the track's file (using juce::File object), title (extracted from the file name) and length (calculated in mm:ss)
 - o **Equality checking:** provide an overloaded equality operator to compare tracks (based on title) to prevent duplicate entries in the PlaylistComponent.
- **DeckGUI:** Updated new buttons for toggling full track loop, manual loop, loop in and loop out. Event listeners added to process button clicks and update DJAudioPlayer accordingly.
- **WaveformDisplay:** Modified to include methods like *setPositionRelative*, *loopRegionSet*, *clearLoopRegion*, and updated *paint()* to render loop region with overlay colours.

Loop Implementation Process:

When manual loop is enabled, user will then press the "Loop In" then "Loop Out" buttons to select loop section. The application records these positions and calls *setManualLoop* on DJAudioPlayer. DJAudioPlayer then uses these stored values in its audio callback (*getNextAudioBlock*) to restart playback at the loop start when the playhead reaches the loop end and repeats this process until manual loop is disabled. The waveform display's *setLoopRegion* method is called to visually update loop area with an overlay.


```

// Looping Controls ** Own Codes **
void DJAudioPlayer::setLooping(bool shouldLoop)
{
    if (shouldLoop)
    {
        loopStart = 0.0;
        loopEnd = transportSource.getLengthInSeconds();
        isLooping = true;
        DBG("Full track loop enabled: 0.0s to " << loopEnd << "s");
    }
    else
    {
        isLooping = false;
        DBG("Full track looping disabled.");
    }
}

void DJAudioPlayer::setManualLoop(double start, double end)
{
    double trackLength = transportSource.getLengthInSeconds();
    if (start >= 0.0 && end > start && end <= trackLength)
    {
        loopStart = start;
        loopEnd = end;
        manualLoopActive = true;
        // Optionally, disable full-track looping when manual loop is active.
        isLooping = false;
        DBG("Manual loop set: " << loopStart << "s to " << loopEnd << "s");
    }
    else
    {
        DBG("DJAudioPlayer::setManualLoop - Invalid loop range!");
    }
}

void DJAudioPlayer::clearManualLoop()
{
    manualLoopActive = false;
    // Optionally reset loop boundaries:
    loopStart = 0.0;
    loopEnd = transportSource.getLengthInSeconds();
    DBG("Manual loop cleared.");
}

void DJAudioPlayer::checkLooping()
{
    if (isLooping && transportSource.getCurrentPosition() >= loopEnd)
    {
        transportSource.setPosition(loopStart);
        transportSource.start(); // Ensure playback resumes
        DBG("Loop restarted (timer): repositioned to " << loopStart);
    }
}

// Draw loop region with overlay (if selected)
if (loopRegionSet && totalTrackLength > 0)
{
    // Calculate the x positions corresponding to loopStart and loopEnd
    int xStart = static_cast<int>((loopStart / totalTrackLength) * getWidth());
    int xEnd = static_cast<int>((loopEnd / totalTrackLength) * getWidth());
    int regionWidth = xEnd - xStart;

    g.setColour(juce::Colour::fromRGB(31, 13, 14).withAlpha(0.3f));
    g.fillRect(xStart, 0, regionWidth, getHeight());
}

// Playback Position and Looping ** Own Codes **
void WaveformDisplay::setPositionRelative(double pos)
{
    if (pos != position && pos==pos)
    {
        position = pos;
        repaint();
    }
}

```

Playlist Implementation Process:

The PlaylistComponent class manages the music library by maintaining a vector of Track objects—each encapsulating a file, title, and duration—while implementing JUCE’s TableListBoxModel for display. When a user imports files via the import button or drag-and-drop, the importTracks() method is called to add new Track instances (after checking for duplicates using isInTracks()) to the collection. The component also provides search and removal functionality and uses saveLibrary() and loadLibrary() to persist the library state in a CSV file between sessions. Finally, when a track is selected, loadInPlayer() is invoked to pass the corresponding Track to the DJAudioPlayer’s loadFile() method, seamlessly integrating the library with the deck controls.

```
// Track Management ** Own Codes **
void PlaylistComponent::loadInPlayer(DeckGUI* deckGUI)
{
    int selectedRow{ library.getSelectedRow() };
    if (selectedRow != -1)
    {
        DBG("Adding: " << tracks[selectedRow].title << " to Player");
        deckGUI->loadFile(tracks[selectedRow].url);
    }
    else
    {
        juce::AlertWindow::showMessageBoxAsync(
            juce::AlertWindow::InfoIcon,
            "Add on to Deck Info:",
            "Select track to add to deck",
            "OK",
            nullptr
        );
    }
}

void PlaylistComponent::importTracks()
{
    DBG("Loading tracks from file...");

    //initialize file chooser
    auto fileChooserFlags = juce::FileBrowserComponent::canSelectFiles | juce::FileBrowserComponent::openMode |
        juce::FileBrowserComponent::canSelectMultipleItems;

    fChooser.launchAsync(fileChooserFlags, [this](const juce::FileChooser& fc)
    {
        auto files = fc.getResults(); // Get selected files
        for (const auto& file : files)
        {
            juce::String fileNameWithoutExtension = file.getFileNameWithoutExtension();
            if (!isInTracks(fileNameWithoutExtension)) // If not already loaded
            {
                Track newTrack{ file };
                juce::URL audioURL{ file };
                newTrack.length = getLength(audioURL);
                tracks.push_back(newTrack);
                DBG("loaded file: " << newTrack.title);
                library.updateContent();
            }
            else // Display info message
            {
                juce::AlertWindow::showMessageBoxAsync(
                    juce::AlertWindow::InfoIcon,
                    "Load information:",
                    fileNameWithoutExtension + " already loaded",
                    "OK"
                );
            }
        }
    });
}
```

```

bool PlaylistComponent::isInTracks(juce::String fileNameWithoutExtension)
{
    return (std::find(tracks.begin(), tracks.end(), fileNameWithoutExtension) != tracks.end());
}

void PlaylistComponent::deleteFromTracks(int id)
{
    tracks.erase(tracks.begin() + id);
}

// Track Length Conversion ** Own Codes **
juce::String PlaylistComponent::getLength(juce::URL audioURL)
{
    playerForParsingMetaData->loadFile(audioURL);
    return secondsToMinutes(playerForParsingMetaData->getLengthInSeconds());
}

juce::String PlaylistComponent::secondsToMinutes(double seconds)
{
    int secondsRounded = static_cast<int>(std::round(seconds));
    return juce::String(secondsRounded / 60) + ":" + juce::String(secondsRounded % 60).paddedLeft('0', 2);
}

// Library Management ** Own Codes **
void PlaylistComponent::searchLibrary(juce::String searchText)
{
    DBG("Searching library for: " << searchText);
    if (searchText != "")
    {
        int rowNumber = whereInTracks(searchText);
        library.selectRow(rowNumber);
    }
    else
    {
        library.deselectAllRows();
    }
}

int PlaylistComponent::whereInTracks(juce::String searchText)
{
    auto it = std::find_if(tracks.begin(), tracks.end(),
        [&searchText](const Track& obj) { return obj.title.contains(searchText); });

    return (it != tracks.end()) ? std::distance(tracks.begin(), it) : -1;
}

void PlaylistComponent::saveLibrary()
{
    // create .csv to save library
    std::ofstream myLibrary("my-library.csv");

    // save library to file
    for (Track& t : tracks)
    {
        myLibrary << t.file.getFullPathName() << "," << t.length << "\n";
    }
}

void PlaylistComponent::loadLibrary()
{
    std::ifstream myLibrary("my-library.csv");
    std::string filePath, length;

    while (getline(myLibrary, filePath, ','))
    {
        Track newTrack{ juce::File{ filePath } };
        getline(myLibrary, length);
        newTrack.length = length;
        tracks.push_back(newTrack);
    }
}

#include "Track.h"

// Constructor
Track::Track(juce::File _file)
    : file(_file),
      title(_file.getFileNameWithoutExtension()),
      url(juce::URL{ _file })
{
    DBG("Created track with title: " << title);
}

// Operator Overloading
bool Track::operator==(const juce::String& other) const
{
    return title == other;
}

```

APPENDIX

A1: CODE

```
*****
*****

1. Main.cpp
*****
*****

/*
=====

=====

This file was auto-generated!
It contains the basic startup code for a JUCE application.
** Most codes are same as course materials **

=====

=====

*/

#include "../JuceLibraryCode/JuceHeader.h"
#include "MainComponent.h"

//
=====

=====

{
class DJStudioApplication : public juce::JUCEApplication
public:
//
=====

=====

DJStudioApplication() {}
const juce::String getApplicationName() override { return ProjectInfo::projectName; }
const juce::String getApplicationVersion() override { return ProjectInfo::versionString; }
bool moreThanOneInstanceAllowed() override { return
true; }
{ return
{ return
//
=====

=====

{
void initialise (const juce::String& commandLine) override
// This method is where you should put your application's
initialisation code..
mainWindow.reset (new MainWindow (getApplicationName()));
}

void shutdown() override
```

```

{ // Add your application's shutdown code here..
mainWindow = nullptr; // (deletes our window)
}
//
=====
=====
{
void systemRequestedQuit() override
// This is called when the app is being asked to quit: you
can ignore this
// request and let the app carry on running, or call quit()
to allow the app to close.
quit();
}
void anotherInstanceStarted (const juce::String& commandLine)
override
{
juce::Logger::writeToLog("Another instance started: " +
commandLine);
}
//
=====
=====
/*
This class implements the desktop window that contains an
instance of
our MainComponent class.
*/
{
class MainWindow : public juce::DocumentWindow
public:
MainWindow(juce::String name) : DocumentWindow(name,
juce::Colour::fromRGB(224, 242, 241),
DocumentWindow::allButtons)
{
setUsingNativeTitleBar (true);
setContentOwned (new MainComponent(), true);
#ifdef JUCE_IOS || JUCE_ANDROID
setFullScreen (true);
#else
setResizable (true, true);
centreWithSize (getWidth(), getHeight());
#endif
setVisible (true);
}

```

```

void closeButtonPressed() override
{is pressed. Here, we'll just
// Closes the application when the window's close button
// ask the app to quit when this happens, but you can
change this to do
// whatever you need.
JUCEApplication::getInstance()->systemRequestedQuit();
}
/* Note: Be careful if you override any DocumentWindow
methods - the base
break its functionality.
instead, but if
make sure your
class uses a lot of them, so by overriding you might
It's best to do all your work in your content component
you really have to override any DocumentWindow methods,
subclass also calls the superclass's method.
*/
private:
JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (MainWindow)
};
private:
std::unique_ptr<MainWindow> mainWindow;
};
//
=====
=====
// This macro generates the main() routine that launches the app.
START_JUCE_APPLICATION (DJStudioApplication)
*****
*****

2. MainComponent.h
*****
*****
#pragma once
#include <JuceHeader.h>
#include "DJAudioplayer.h"
#include "DeckGUI.h"
#include "PlaylistComponent.h"
//
=====
=====
{
class MainComponent : public juce::AudioAppComponent
public:

```

```

//
=====

=====//
=====

MainComponent();
~MainComponent() override;
=====

//
=====

=====

void prepareToPlay (int samplesPerBlockExpected, double
sampleRate) override;
void getNextAudioBlock (const juce::AudioSourceChannelInfo&
bufferToFill) override;
void releaseResources() override;

//
=====

=====

void paint (juce::Graphics& g) override;
void resized() override;
private:
//
=====

=====

// Your private member variables go here...
juce::AudioFormatManager formatManager;
juce::AudioThumbnailCache thumbCache{100};
DJAudioPlayer player1{formatManager};
DeckGUI deckGUI1{&player1, formatManager, thumbCache};
DJAudioPlayer player2{formatManager};
DeckGUI deckGUI2{&player2, formatManager, thumbCache };
DJAudioPlayer playerForParsingMetaData{ formatManager }; // Own
Code
juce::MixerAudioSource mixerSource;
PlaylistComponent playlistComponent{ &deckGUI1, &deckGUI2,
&playerForParsingMetaData };
JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (MainComponent)
};

*****

*****

3. MainComponent.cpp

*****

*****

#include "MainComponent.h"

//

```

```

=====
=====//
=====
MainComponent::MainComponent()
{
=====

// Setting size of main component
setSize (800, 600);

// Request audio recording permissions if required
if (juce::RuntimePermissions::isRequired
(juce::RuntimePermissions::recordAudio)
&& ! juce::RuntimePermissions::isGranted
(juce::RuntimePermissions::recordAudio))
{
juce::RuntimePermissions::request
(juce::RuntimePermissions::recordAudio,
[&] (bool granted)
{ setAudioChannels (granted ? 2 : 0, 2); });
}
else
{
// Set up 2 audio channels (no input, 2 outputs)
setAudioChannels (0, 2);
}

// Add child components to main UI
addAndMakeVisible(deckGUI1);
addAndMakeVisible(deckGUI2);
addAndMakeVisible(playlistComponent);

// Register audio formats
formatManager.registerBasicFormats();
}

//
=====

=====
// Destructor
MainComponent::~MainComponent()
{
// Shut down the audio device and clears the audio sources
shutdownAudio();
}

//
=====

=====
// Audio Processing Methods
void MainComponent::prepareToPlay (int samplesPerBlockExpected,

```



```

double sampleRate)
{
// Audio Players
player1.prepareToPlay(samplesPerBlockExpected, sampleRate);
player2.prepareToPlay(samplesPerBlockExpected, sampleRate);
// Mixer Source
mixerSource.prepareToPlay(samplesPerBlockExpected, sampleRate);mixerSource.addInputSource(&player1, false);
mixerSource.addInputSource(&player2, false);
}
void MainComponent::getNextAudioBlock(const
juce::AudioSourceChannelInfo& bufferToFill)
{
// Pass audio to mixer source
mixerSource.getNextAudioBlock(bufferToFill);
}
{
void MainComponent::releaseResources()
// Release resources for players and mixer
player1.releaseResources();
player2.releaseResources();
mixerSource.releaseResources();
}
//
=====

=====

// UI Rendering methods ** Layout are own codes **
void MainComponent::paint (juce::Graphics& g)
{
// Fill background with Soft Muted Teal (Light Aqua)
g.fillAll(juce::Colour::fromRGB(224, 242, 241));
}
{
void MainComponent::resized()
// Layout playlist component in top half of component
playlistComponent.setBounds(0, 0, getWidth(), getHeight() / 2);
// Layout deck GUIs in the bottom half.
int halfWidth = getWidth() / 2;
int halfHeight = getHeight() / 2;
deckGUI1.setBounds(0, halfHeight, halfWidth, halfHeight);
deckGUI2.setBounds(halfWidth, halfHeight, halfWidth,
halfHeight);
}
*****
*****

```

4. DeckGUI.h

```

*****
*****

/*
=====

=====

DeckGUI.hCreated: 12 Feb 2025 9:59:11pm
Author: Devon Xavior
=====

=====

*/

#pragma once
#include <JuceHeader.h>
#include "DJAudioPlayer.h"
#include "WaveformDisplay.h"
//
=====

=====

// Custom LookAndFeel for Knobs ** Own Codes **
class CustomKnobLookAndFeel : public juce::LookAndFeel_V4
{
public:
width, int height,
void drawRotarySlider(juce::Graphics& g, int x, int y, int
float sliderPosProportional, float
rotaryStartAngle,
float rotaryEndAngle, juce::Slider&
slider) override
{
// Define knob properties
auto radius = juce::jmin(width / 2, height / 2) - 10.0f;
auto centerX = x + width * 0.5f;
auto centerY = y + height * 0.45f;
auto angle = rotaryStartAngle + sliderPosProportional *
(rotaryEndAngle - rotaryStartAngle);
// Draw knob outline with a thicker stroke
g.setColour(juce::Colour::fromRGB(178, 132, 109));
g.drawEllipse(centerX - radius, centerY - radius, radius *
2, radius * 2, 6.0f);
// Draw knob background
g.setColour(juce::Colour::fromRGB(153, 204, 204));
g.fillEllipse(centerX - radius + 1, centerY - radius + 1,
(radius * 2) - 2, (radius * 2) - 2);
// Draw knob marker (indicator line)
g.setColour(juce::Colour::fromRGB(178, 132, 109));
juce::Path marker;

```

```

marker.startNewSubPath(centerX, centerY);
marker.lineTo(centerX + radius * 0.9f * std::cos(angle),
centerY + radius * 0.9f * std::sin(angle));
g.strokePath(marker, juce::PathStrokeType(2.5f));
} // Main slider layout
juce::Slider::SliderLayout getSliderLayout(juce::Slider& slider)
override
{
juce::Slider::SliderLayout layout =
LookAndFeel_V4::getSliderLayout(slider);
layout.textBoxBounds.setY(layout.textBoxBounds.getY() -
8);
return layout;
}
// Slider text box
juce::Label* createSliderTextBox(juce::Slider& slider) override
{
auto* textBox =
LookAndFeel_V4::createSliderTextBox(slider);
textBox->setColour(juce::Label::textColourId,
juce::Colours::darkgrey);
return textBox;
}
};
// DeckGUI Class
class DeckGUI : public juce::Component,
public juce::Button::Listener,
public juce::Slider::Listener,
public juce::FileDragAndDropTarget,
public juce::Timer
{
public:
// Constructor and Destructor
DeckGUI(DJAudioPlayer* player,
juce::AudioFormatManager & formatManagerToUse,
juce::AudioThumbnailCache & cacheToUse);
~DeckGUI() override;
// Component Lifecycle
void paint (juce::Graphics&) override;
void resized() override;
// Event Listeners
void buttonClicked(juce::Button*) override;
void sliderValueChanged(juce::Slider*) override;
// Drag-and-Drop support
bool isInterestedInFileDrag(const juce::StringArray& files)

```

```

override;
void filesDropped(const juce::StringArray& files, int x, int y)
override;
// Timer Callback (for waveform updates)
void timerCallback() override;private:
// Playback controls
juce::TextButton playButton{ "PLAY" };
juce::TextButton stopButton{ "STOP" };
juce::TextButton loadButton{ "Select File" };
// Looping controls ** Own Codes **
juce::TextButton loopInButton { "LOOP IN" };
juce::TextButton loopOutButton { "LOOP OUT" };
juce::TextButton fullLoopToggleButton{ "Full Loop: Off" };
juce::TextButton manualLoopToggleButton{ "Manual Loop: Off" };
double loopStart = 0.0;
double loopEnd = 0.0;
bool fullLoopOn = false;
bool manualLoopMode = false;
// Sliders and Controls
juce::Slider VolSlider;
juce::Slider speedSlider;
juce::Slider posSlider;
// CustomLookAndFeel for speed slider knob ** Own Codes **
CustomKnobLookAndFeel speedKnobLookAndFeel;
// Custom colour
juce::Colour warmSand = juce::Colour::fromRGB(178, 132, 109);
// File Handling
juce::FileChooser fChooser{"Select a file..."};
// Labels
juce::Label VolLabel;
juce::Label speedLabel;
// Utility and Processing
void loadFile(juce::URL audioURL);
DJAudioPlayer* player;
//Waveform visual
WaveformDisplay waveformDisplay;
juce::SharedResourcePointer< juce::TooltipWindow >
sharedTooltip; // ** Own Codes **
friend class PlaylistComponent;
JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (DeckGUI)
};
*****
*****

5. DeckGUI.cpp*****
*****

```

```

/*
=====

=====
DeckGUI.cpp
Created: 12 Feb 2025 9:59:11pm
Author: Devon Xavior
=====

=====

*/
#include <JuceHeader.h>
#include "DeckGUI.h"
//
=====

=====
// Constructor
DeckGUI::DeckGUI(DJAudioPlayer* _player,
juce::AudioFormatManager& formatManagerToUse,
juce::AudioThumbnailCache& cacheToUse) :
player(_player),
waveformDisplay(formatManagerToUse, cacheToUse)
{
// Add and make visible UI components
addAndMakeVisible(playButton);
addAndMakeVisible(stopButton);
// Loop Buttons ** Own Codes **
addAndMakeVisible(fullLoopToggleButton);
addAndMakeVisible(manualLoopToggleButton);
addAndMakeVisible(loopInButton);
addAndMakeVisible(loopOutButton);
addAndMakeVisible(loadButton);
addAndMakeVisible(VolSlider);
addAndMakeVisible(VolLabel); // ** Own Codes **
addAndMakeVisible(speedSlider);
addAndMakeVisible(speedLabel); // ** Own Codes **
addAndMakeVisible(posSlider);
addAndMakeVisible(waveformDisplay);
// Add button listeners
playButton.addListener(this);
stopButton.addListener(this);
loadButton.addListener(this); // ** Own Codes **
fullLoopToggleButton.addListener(this);
manualLoopToggleButton.addListener(this);
loopInButton.addListener(this);
loopOutButton.addListener(this);
// Add slider listeners

```

```

VolSlider.addListener(this);
speedSlider.addListener(this);
posSlider.addListener(this);
// Position slider colors ** Own Codes **
getLookAndFeel().setColour(juce::Slider::thumbColourId,
juce::Colour::fromRGB(178, 132, 109));
getLookAndFeel().setColour(juce::Slider::trackColourId,
juce::Colour::fromRGB(195, 221, 218));
// Position Slider Configuration ** Own Codes **
posSlider.setRange(0.0, 1.0);
posSlider.setNumDecimalPlacesToDisplay(2);
posSlider.setSliderStyle(juce::Slider::SliderStyle::LinearHorizontal
);
posSlider.setTextBoxStyle(juce::Slider::NoTextBox, false, 0, 0);
// Volume Slider Configuration ** Own Codes **
VolSlider.setRange(0.0, 1.0);
VolSlider.setNumDecimalPlacesToDisplay(2);
VolSlider.setValue(0.5); //default volume half of max vol
VolSlider.setSliderStyle(juce::Slider::SliderStyle::LinearBar);
VolSlider.setTextBoxStyle(juce::Slider::NoTextBox, false, 0, 0);
// Volume Label Configuration ** Own Codes **
VolLabel.setText("Volume", juce::dontSendNotification);
VolLabel.attachToComponent(&VolSlider, false);
VolLabel.setJustificationType(juce::Justification::centred);
VolLabel.setColour(juce::Label::textColourId,
juce::Colours::black);
// Speed Label Configuration ** Own Codes **
speedLabel.setText("Speed", juce::dontSendNotification);
speedLabel.attachToComponent(&speedSlider, false);
speedLabel.setJustificationType(juce::Justification::centred);
speedLabel.setColour(juce::Label::textColourId,
juce::Colours::black);
// Speed Knob Configuration ** Own Codes **
speedSlider.setSliderStyle(juce::Slider::RotaryHorizontalDrag);
speedSlider.setTextBoxStyle(juce::Slider::TextBoxBelow, false,
50, 20);
(slow) to 3x (fast)
speedSlider.setRange(0.5, 3.0, 0.01); // Speed range from 0.5x
speedSlider.setValue(1.0); // Default speed at 1.0x (normal
speed)speedSlider.setMouseDragSensitivity(150);
speedSlider.setLookAndFeel(&speedKnobLookAndFeel);
startTimer(500);
}
// Destructor
DeckGUI::~~DeckGUI()

```

```

{
stopTimer();
}
// UI Rendering ** Own Codes **
void DeckGUI::paint (juce::Graphics& g)
{
g.fillAll (juce::Colour::fromRGB(224, 242, 241)); // Soft Muted
Teal background (Light Aqua)
g.setColour (juce::Colour::fromRGB(178, 132, 109)); // Warm Sand
outline (Soft Copper)
g.drawRect (getLocalBounds(), 2); // Slightly thicker outline
for a refined look
loadButton.setColour(juce::TextButton::buttonColourId,
warmSand);
warmSand);
warmSand);
playButton.setColour(juce::TextButton::buttonColourId,
stopButton.setColour(juce::TextButton::buttonColourId,
}
// Button Click Events ** Own Codes **
void DeckGUI::buttonClicked(juce::Button* button)
{
if (button == &playButton)
{
std::cout << "Play button was clicked " << std::endl;
player->start();
}
{
if (button == &stopButton)
std::cout << "Stop button was clicked " << std::endl;
player->stop();
}
{
if (button == &fullLoopToggleButton)
// Toggle full-track loop mode.
fullLoopOn = !fullLoopOn;
player->setLooping(fullLoopOn); // This method loops the
entire track.
fullLoopToggleButton.setButtonText(fullLoopOn ? "Full Loop:
On" : "Full Loop: Off");// When full-loop is enabled, disable manual loop mode.
if (fullLoopOn)
{
manualLoopMode = false;
manualLoopToggleButton.setButtonText("Manual Loop:
Off");

```

```

}
}
{
if (button == &manualLoopToggleButton)
manualLoopMode = !manualLoopMode;
manualLoopToggleButton.setText(manualLoopMode ?
"Manual Loop: On" : "Manual Loop: Off");
if (!manualLoopMode)
{
// When turning off manual loop mode, clear the manual
loop in the player.
player->clearManualLoop();
waveformDisplay.clearLoopRegion();
}
else
{
// When manual loop is enabled, disable full-track
looping if it's active.
fullLoopOn = false;
fullLoopToggleButton.setText("Full Loop: Off");
// Optionally, clear any previously set manual loop.
player->clearManualLoop();
}
}
{
if (button == &loopInButton)
if (manualLoopMode)
{
double pos = player->getCurrentPosition();
loopStart = pos;
DBG("Loop In set at: " << loopStart << " seconds");
// Optionally update the waveform display to show only
the start until out is set:
waveformDisplay.setLoopRegion(loopStart, loopEnd); //
loopEnd may be 0 if not set yet
}
else
{
DBG("Manual loop mode is off. Enable it to set loop
points.");
}
}
{
if (button == &loopOutButton)
if (manualLoopMode)

```



```

{active."});
double pos = player->getCurrentPosition();
loopEnd = pos;
if (loopEnd > loopStart)
{
player->setManualLoop(loopStart, loopEnd);
DBG("Loop Out set at: " << loopEnd << ". Manual loop
waveformDisplay.setLoopRegion(loopStart, loopEnd);
}
else
{
DBG("Invalid manual loop: Loop Out must be after
Loop In.");
}
}
else
{
DBG("Manual loop mode is off. Enable it to set loop
points.");
}
}
{
if (button == &loadButton)
auto fileChooserFlags =
juce::FileBrowserComponent::canSelectFiles;
fChooser.launchAsync(fileChooserFlags, [this](const
juce::FileChooser& chooser)
{
player->loadFile(juce::URL{chooser.getResult()});
// and now the waveformDisplay as well
waveformDisplay.loadURL(juce::URL{chooser.getResult()});
double length = player->getLengthInSeconds();
waveformDisplay.setTrackLength(length);
});
}
}
// Slider Value Changed Events
void DeckGUI::sliderValueChanged(juce::Slider* slider)
{
if (slider == &VolSlider)
{
player->setGain(slider->getValue());
}
}
{
if (slider == &speedSlider)

```

```

player->setSpeed(slider->getValue());
}
{
if (slider == &posSlider)
player->setPositionRelative(slider->getValue());
}
} // File Drag and Drop Events
bool DeckGUI::isInterestedInFileDrag(const juce::StringArray &files)
{
DBG("DeckGUI::isInterestedInFileDrag");
return true;
}
void DeckGUI::filesDropped(const juce::StringArray &files, int x,
int y) // ** Own Codes **
{
DBG("DeckGUI::filesDropped");
if (files.size() == 1)
{
juce::File droppedFile(files[0]);
juce::URL fileURL(droppedFile);
player->loadFile(fileURL);
waveformDisplay.loadURL(fileURL);
}
}
// File Load Function
void DeckGUI::loadFile(juce::URL audioURL)
{
DBG("DeckGUI::loadFile called");
player->loadFile(audioURL);
waveformDisplay.loadURL(audioURL);
}
// Component Resizing ** Own Codes **
void DeckGUI::resized()
{
double rowH = getHeight() / 6;
double colW = getWidth() / 4;
waveformDisplay.setBounds(0, 0, getWidth(), rowH * 2);
posSlider.setBounds(0, rowH * 2, getWidth(), rowH);
VolSlider.setBounds(colW * 2 - 20, rowH * 3 + 10, 60, rowH * 2 -
10);
speedSlider.setBounds(colW * 3 - 30, rowH * 3 + 10, 80, rowH * 2
- 10);
playButton.setBounds(colW/8 + 10, rowH * 5 + 10, colW - 10 ,
rowH - 20);
- 20);

```

```

rowH - 20);
stopButton.setBounds(colW + 30 , rowH * 5 + 10, colW - 10, rowH
loadButton.setBounds(colW*2 + 40 , rowH * 5 + 10, colW + 10,
fullLoopToggleButton.setBounds(colW - 60, rowH * 3 - 5, 100,
20);
manualLoopToggleButton.setBounds(colW - 60, rowH * 3 + 20, 100,
20);loopInButton.setBounds(colW - 65, rowH * 3 + 55, 50, 40);
loopOutButton.setBounds(colW - 5, rowH * 3 + 55, 50, 40);
}
// Timer Callback (Waveform Position Update) ** Own Codes **
void DeckGUI::timerCallback()
{
player->checkLooping();
double relPos = player->getPositionRelative();
// Check for NaN or non-finite values
if (std::isnan(relPos) || !std::isfinite(relPos))
{
relPos = 0.0;
}
relPos = juce::jlimit(0.0, 1.0, relPos);
if (relPos > 0.9999)
relPos = 1.0;
else
relPos = std::round(relPos * 1000.0) / 1000.0;
waveformDisplay.setPositionRelative(relPos);
posSlider.setValue(relPos, juce::dontSendNotification);
}

```

6.DJAudioPlayer.h

/*

=====

=====

DJAudioPlayer.h

Created: 12 Feb 2025 9:59:26pm

Author: Devon Xavier

=====

=====

*/

#pragma once

#include <JuceHeader.h>

// DJ AuddioPlayer Class

class DJAudioPlayer : public juce::AudioSource

```

{public:
// Constructor and Destructor
DJAudioPlayer(juce::AudioFormatManager& _formatManager);
~DJAudioPlayer() override;
// AudioSource Overrides
void prepareToPlay(int samplesPerBlockExpected, double
sampleRate) override;
void getNextAudioBlock(const juce::AudioSourceChannelInfo&
bufferToFill) override;
void releaseResources() override;
// Audio Playback Controls
void loadFile(juce::URL audioURL);
void setGain(double gain);
void setSpeed(double ratio);
void setPosition(double posInSecs);
void setPositionRelative(double pos);
void start();
void stop();
// Looping Functionality ** Own Codes **
void setLooping(bool shouldLoop);
void checkLooping();
void setManualLoop(double start, double end);
void clearManualLoop();
// Utility Functions ** Own Codes **
double getPositionRelative();
double getLengthInSeconds();
double getCurrentPosition();
private:
// Audio Processing Components
juce::AudioFormatManager& formatManager;
std::unique_ptr<juce::AudioFormatReaderSource> readerSource;
juce::AudioTransportSource transportSource;
juce::ResamplingAudioSource resampleSource{&transportSource,
false, 2};
// Looping Variables ** Own Codes **
bool isLooping = false;
bool manualLoopActive = false;
double loopStart = 0.0;
double loopEnd = 0.0;
};

```

```

*****

```

```

*****

```

7. DJAudioPlayer.cpp

```

*****

```

```

*****

```

```

/*=====
=====
DJAudioPlayer.cpp
Created: 12 Feb 2025 9:59:26pm
Author: Devon Xavier
=====
=====
*/

#include "DJAudioPlayer.h"
// Constructor and Destructor
DJAudioPlayer::DJAudioPlayer(juce::AudioFormatManager&
_formatManager):formatManager(_formatManager)
{
}
DJAudioPlayer::~DJAudioPlayer()
{
}
// AudioSource Overrides
void DJAudioPlayer::prepareToPlay(int samplesPerBlockExpected,
double sampleRate)
{
transportSource.prepareToPlay(samplesPerBlockExpected,
sampleRate);
sampleRate);
resampleSource.prepareToPlay(samplesPerBlockExpected,
}
void DJAudioPlayer::getNextAudioBlock(const
juce::AudioSourceChannelInfo& bufferToFill) // ** Own Codes **
{
// Manual loop takes precedence over full-track looping
if (manualLoopActive && transportSource.getCurrentPosition()
>= loopEnd)
{
transportSource.setPosition(loopStart);
if (!transportSource.isPlaying())
transportSource.start();
DBG("Manual loop restarted: repositioned to " <<
loopStart);
}
>= loopEnd)
{
else if (isLooping && transportSource.getCurrentPosition()
transportSource.setPosition(loopStart);
if (!transportSource.isPlaying())
transportSource.start();DBG("Full track loop restarted.");
}

```

```

}
resampleSource.getNextAudioBlock(bufferToFill);
}
{
void DJAudioPlayer::releaseResources()
transportSource.releaseResources();
resampleSource.releaseResources();
}
// Audio Playback Controls ** Own Codes **
void DJAudioPlayer::loadFile(juce::URL audioURL)
{
auto* reader =
formatManager.createReaderFor(audioURL.createInputStream(false));
if (reader != nullptr)
{
std::unique_ptr<juce::AudioFormatReaderSource> newSource
(new juce::AudioFormatReaderSource (reader,
true));
transportSource.setSource (newSource.get(), 0, nullptr,
reader->sampleRate);
readerSource.reset (newSource.release());
}
}
{
void DJAudioPlayer::setGain(double gain)
if (gain < 0.0||gain > 1.0)
{
DBG("DJAudioPlayer::setGain gain should be between 0 and
1");
}
else {
}
transportSource.setGain(gain);
}
{
void DJAudioPlayer::setSpeed(double ratio)
if (ratio <= 0 || ratio > 100.0)
{
DBG("DJAudioPlayer::setSpeed ratio should be between 0 and
100");
}
else {
resampleSource.setResamplingRatio(ratio);
}
}
}

```

```

void DJAudioPlayer::setPosition(double posInSecs)
{transportSource.setPosition(posInSecs);
}
{
void DJAudioPlayer::setPositionRelative(double pos)
if (pos < 0.0 || pos > 1.0)
{
DBG("DJAudioPlayer::setPositionRelative pos should be
between 0 and 1");
}
else {
double posInSecs = transportSource.getLengthInSeconds() *
pos;
setPosition(posInSecs);
}
double DJAudioPlayer::getPositionRelative()
return transportSource.getCurrentPosition() /
transportSource.getLengthInSeconds();
double DJAudioPlayer::getLengthInSeconds()
return transportSource.getLengthInSeconds();
double DJAudioPlayer::getCurrentPosition() // ** Own Codes **
return transportSource.getCurrentPosition();
}
{
}
{
}
{
}
{
}
{
void DJAudioPlayer::start()
if (transportSource.getCurrentPosition() >=
transportSource.getLengthInSeconds() - 0.01)
{
transportSource.setPosition(0.0);
}
transportSource.start();
}
void DJAudioPlayer::stop()
{
transportSource.stop();
}
// Looping Controls ** Own Codes **
void DJAudioPlayer::setLooping(bool shouldLoop)
{

```

```

if (shouldLoop)
{
loopStart = 0.0; loopEnd = transportSource.getLengthInSeconds();
isLooping = true;
DBG("Full track loop enabled: 0.0s to " << loopEnd << "s");
}
else
{
isLooping = false;
DBG("Full track looping disabled.");
}
}

{
void DJAudioPlayer::setManualLoop(double start, double end)
double trackLength = transportSource.getLengthInSeconds();
if (start >= 0.0 && end > start && end <= trackLength)
{
loopStart = start;
loopEnd = end;
manualLoopActive = true;
// Optionally, disable full-track looping when manual loop
is active.
isLooping = false;
DBG("Manual loop set: " << loopStart << "s to " << loopEnd
<< "s");
}
else
{
}
DBG("DJAudioPlayer::setManualLoop - Invalid loop range!");
}

{
void DJAudioPlayer::clearManualLoop()
manualLoopActive = false;
// Optionally reset loop boundaries:
loopStart = 0.0;
loopEnd = transportSource.getLengthInSeconds();
DBG("Manual loop cleared.");
}

{
void DJAudioPlayer::checkLooping()
if (isLooping && transportSource.getCurrentPosition() >=
loopEnd)
{
transportSource.setPosition(loopStart);
}
}

```



```

transportSource.start(); // Ensure playback resumes
DBG("Loop restarted (timer): repositioned to " <<
loopStart);
}
}

*****
*****8. WaveformDisplay.h
*****
*****

/*
=====

=====

WaveformDisplay.h
Created: 12 Feb 2025 9:59:45pm
Author: Devon Xavior

=====

=====

*/

#pragma once
#include <JuceHeader.h>
#include <string>
//
=====

=====

// WaveformDisplay Class
class WaveformDisplay : public juce::Component,
public juce::ChangeListener
{
public:
// Constructor and Deconstructor
WaveformDisplay(juce::AudioFormatManager& formatManagerToUse,
juce::AudioThumbnailCache& cacheToUse);
~WaveformDisplay() override;
// Component Lifecycle
void paint (juce::Graphics&) override;
// Event Handling
void changeListenerCallback(juce::ChangeBroadcaster* source)
override;
// Audio File Management
void loadURL(juce::URL audioURL);
// Playback and Looping ** Own Codes **
void setPositionRelative(double pos);
void setTrackLength(double newLength)
{
totalTrackLength = newLength;

```

```

repaint();
}void setLoopRegion(double start, double end)
{
loopStart = start;
loopEnd = end;
loopRegionSet = (end > start);
repaint();
}
{
void clearLoopRegion()
loopRegionSet = false;
repaint();
}
private:
// Internal State Variables
juce::AudioThumbnail audioThumb;
bool fileLoaded = false;
double position = 0.0;
std::string nowPlaying;
// Track and Looping data ** Own Codes **
double totalTrackLength = 0.0;
double loopStart = 0.0;
double loopEnd = 0.0;
bool loopRegionSet = false;
JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (WaveformDisplay)
};

```

```

*****
*****

```

9. WaveformDisplay.cpp

```

*****
*****

```

```
/*
```

```
=====
```

```
=====
```

WaveformDisplay.cpp

Created: 12 Feb 2025 9:59:45pm

Author: Devon Xavior

```
=====
```

```
=====
```

```
*/
```

```
#include <JuceHeader.h>
```

```
#include "WaveformDisplay.h"
```

```
#include "DJAudioPlayer.h"//
```

```
=====
```

```
=====
```

```

WaveformDisplay::WaveformDisplay(juce::AudioFormatManager&
formatManagerToUse,
juce::AudioThumbnailCache&
cacheToUse)
: audioThumb(1000, formatManagerToUse, cacheToUse),
fileLoaded(false),
position(0)
audioThumb.addChangeListener(this);
{
}
WaveformDisplay::~~WaveformDisplay()
{
}
// Component Rendering ** Own Codes **
void WaveformDisplay::paint (juce::Graphics& g)
{
// Set the background colour to soft teal
g.fillAll(juce::Colour::fromRGB(153, 204, 204));
// Draw component border
g.setColour(juce::Colour::fromRGB(178, 132, 109));
g.drawRect(getLocalBounds(), 2);
if (fileLoaded)
{
// Draw waveform in warm sand colour
g.setColour(juce::Colour::fromRGB(178, 132, 109));
audioThumb.drawChannel(g, getLocalBounds(), 0,
audioThumb.getTotalLength(), 0, 1.0f);
// Draw the position marker in white
g.setColour(juce::Colours::white);
g.fillRect(position * getWidth(), 0, 3, getHeight());
// Display track name
g.setColour(juce::Colours::floralwhite);
g.setFont(16.0f);
g.drawText(nowPlaying, getLocalBounds(),
juce::Justification::centred, true);
}
else {
// Draw placeholder text in white (if no track loaded)
g.setColour(juce::Colours::white);
g.setFont(25.0f);
g.drawText("File not Loaded", getLocalBounds(),
juce::Justification::centred, true);
}
// Draw loop region with overlay (if selected)if (loopRegionSet && totalTrackLength > 0)
{

```

```

// Calculate the x positions corresponding to loopStart and
loopEnd
int xStart = static_cast<int>((loopStart / totalTrackLength)
* getWidth());
* getWidth();
int xEnd = static_cast<int>((loopEnd / totalTrackLength)
int regionWidth = xEnd - xStart;
g.setColour(juce::Colour::fromRGB(31, 13,
14).withAlpha(0.3f));
g.fillRect(xStart, 0, regionWidth, getHeight());
}
}
// Audio File Loading ** Own Codes **
void WaveformDisplay::loadURL(juce::URL audioURL)
{
// Clear previous waveform data and loop region
audioThumb.clear();
clearLoopRegion();
juce::File file = audioURL.getLocalFile();
if (!file.existsAsFile())
{
DBG("WaveformDisplay::loadURL - File does not exist: " <<
file.getFullPathName());
fileLoaded = false;
repaint();
return;
}
else
{
DBG("WaveformDisplay::loadURL - File exists: " <<
file.getFullPathName());
}
// Load track into waveform display
fileLoaded = audioThumb.setSource(new
juce::FileInputSource(file));
if (fileLoaded)
{
DBG("WaveformDisplay::LOADED!");
// Extract file name for display
std::string justFile =
audioURL.toString(false).toStdString();
std::size_t startFilePos = justFile.find_last_of("/");
std::size_t startExtPos = justFile.find_last_of(".");
std::string fileName = justFile.substr(startFilePos + 1,
startExtPos - startFilePos - 1);

```

```

nowPlaying = fileName;// Use getTotalLength() to set the track length.
totalTrackLength = audioThumb.getTotalLength();
DBG("Total track length: " << totalTrackLength);
}
else
{
DBG("WaveformDisplay::NOT LOADED!");
repaint();
}
}
// Event Handling
void
WaveformDisplay::changeListenerCallback(juce::ChangeBroadcaster*
source)
{
DBG("WaveformDisplay::CHANGE RECEIVED!");
repaint();
}
// Playback Position and Looping ** Own Codes **
void WaveformDisplay::setPositionRelative(double pos)
{
if (pos != position && pos==pos)
{
position = pos;
repaint();
}
}

*****

*****

10. PlaylistComponent.h

*****

*****

#pragma once
#include <JuceHeader.h>
#include <vector>
#include <string>
#include <algorithm>
#include <fstream>
#include <functional>
#include "Track.h"
#include "DeckGUI.h"
#include "DJAudioplayer.h"

//
=====
=====

```

```

// PlaylistComponent ** Own Codes **
class PlaylistComponent : public juce::Component,
public juce::TableListBoxModel, public juce::Button::Listener,
public juce::TextEditor::Listener
{
public:
// Constructor and Deconstructor ** Own Codes **
PlaylistComponent(DeckGUI* _deckGUI1,
DeckGUI* _deckGUI2,
DJAudioplayer* _playerForParsingMetaData);
~PlaylistComponent() override;
// Component overrides
void paint(juce::Graphics&) override;
void resized() override;
// TableListBoxModel overrides
int getNumRows() override;
void paintRowBackground(juce::Graphics&, int rowNumber, int
width, int height, bool rowsSelected) override;
void paintCell(juce::Graphics&, int rowNumber, int columnId, int
width, int height, bool rowsSelected) override;
juce::Component* refreshComponentForCell(int rowNumber,
int columnId,
bool isRowSelected,
Component*
existingComponentToUpdate) override;
// Event Listeners
void buttonClicked(juce::Button* button) override;
private:
// UI Components ** Own Codes **
juce::TextButton importButton{ "Import Tracks" };
juce::TextEditor searchField;
juce::TableListBox library;
juce::TextButton addToDeck1Button{ "Add to Deck 1" };
juce::TextButton addToDeck2Button{ "Add to Deck 2" };
juce::FileChooser fChooser{"Select a file..."};
// Track Management ** Own Codes **
std::vector<Track> tracks;
DeckGUI* deckGUI1;
DeckGUI* deckGUI2;
DJAudioplayer* playerForParsingMetaData;
// UI Theme ** Own Codes **
juce::Colour warmSand = juce::Colour::fromRGB(178, 132, 109);
// Helper Functions ** Own Codes **
juce::String getLength(juce::URL audioURL);
juce::String secondsToMinutes(double seconds);

```

```

void importTracks();
void searchLibrary(juce::String searchText);void saveLibrary();
void loadLibrary();
void deleteFromTracks(int id);
bool isInTracks(juce::String fileNameWithoutExtension);
int whereInTracks(juce::String searchText);
void loadInPlayer(DeckGUI* deckGUI);
JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(PlaylistComponent)
};

```

```

*****

```

```

*****

```

11. PlaylistComponent.cpp

```

*****

```

```

*****

```

```

/*

```

```

=====

```

```

=====

```

PlaylistComponent.cpp

Created: 12 Feb 2025 10:00:01pm

Author: Devon Xavier

**** Most codes wrote without assistance ****

```

=====

```

```

=====

```

```

*/

```

```

#include <JuceHeader.h>

```

```

#include "PlaylistComponent.h"

```

```

#include "Track.h"

```

```

//Constructor and Deconstructor ** Own Codes **

```

```

PlaylistComponent::PlaylistComponent (DeckGUI* _deckGUI1,

```

```

DeckGUI* _deckGUI2,

```

```

DJAudioplayer*

```

```

_playerForParsingMetaData)

```

```

: deckGUI1(_deckGUI1),

```

```

deckGUI2(_deckGUI2),

```

```

playerForParsingMetaData(_playerForParsingMetaData)

```

```

{

```

```

// Add UI components ** Own Codes **

```

```

addAndMakeVisible(importButton);

```

```

addAndMakeVisible(searchField);

```

```

addAndMakeVisible(library);

```

```

addAndMakeVisible(addToDeck1Button);

```

```

addAndMakeVisible(addToDeck2Button);

```

```

// Set up listeners ** Own Codes **

```

```

importButton.addListener(this);

```

```

searchField.addListener(this);addToDeck1Button.addListener(this);

```

```

addToDeck2Button.addListener(this);
// Configure search field appearance and placeholder text ** Own
Codes **
searchField.setTextToShowWhenEmpty("Type track title then press
ENTER to search Playlist", juce::Colours::grey);
searchField.onReturnKey = [this]
{ searchLibrary(searchField.getText()); };
// Set up table columns ** Own Codes **
library.setModel(this);
library.getHeader().addColumn("Title", 1, 200);
library.getHeader().addColumn("Duration", 2, 100);
library.getHeader().addColumn("Actions", 3, 100);
// Load any saved library from disk
loadLibrary();
}
PlaylistComponent::~PlaylistComponent()
{
saveLibrary();
}
// UI Rendering ** Own Codes **
void PlaylistComponent::paint(juce::Graphics& g)
{
g.fillAll(juce::Colours::fromRGB(195, 221, 218));
// Set button colours
addToDeck1Button.setColour(juce::TextButton::buttonColourId,
warmSand);
addToDeck2Button.setColour(juce::TextButton::buttonColourId,
warmSand);
importButton.setColour(juce::TextButton::buttonColourId,
warmSand);
}
{
void PlaylistComponent::resized()
auto area = getLocalBounds();
int controlHeight = 40;
// Layout top controls
importButton.setBounds(area.removeFromTop(controlHeight).reduced(5))
;
searchField.setBounds(area.removeFromTop(controlHeight).reduced(5));
// Allocate most space to the table
library.setBounds(area.removeFromTop(area.getHeight() -
controlHeight * 2).reduced(5)); // Layout bottom buttons
auto bottomArea = area.reduced(5);
addToDeck1Button.setBounds(bottomArea.removeFromLeft(bottomArea.getW
idth() / 2).reduced(2));

```



```

addToDeck2Button.setBounds(bottomArea.reduced(2));
}

// Table Handling ** Own Codes **
int PlaylistComponent::getNumRows()
{
return static_cast<int>(tracks.size());
}

void PlaylistComponent::paintRowBackground(juce::Graphics& g, int
rowNumber, int width, int height, bool rowsSelected)
{
g.fillAll(rowsSelected ? juce::Colours::lightblue :
juce::Colours::white);
}

{
void PlaylistComponent::paintCell(juce::Graphics& g, int rowNumber,
int columnId, int width, int height, bool rowsSelected)
if (rowNumber < getNumRows())
{
if (columnId == 1)
{
g.drawText(tracks[rowNumber].title, 2, 0, width - 4,
height, juce::Justification::centredLeft, true);
}
{
if (columnId == 2)
g.drawText(tracks[rowNumber].length, 2, 0, width - 4,
height, juce::Justification::centred, true);
}
}
}

juce::Component* PlaylistComponent::refreshComponentForCell(int
rowNumber, int columnId, bool isRowSelected, juce::Component*
existingComponentToUpdate)
{
if (columnId == 3)
{
if (existingComponentToUpdate == nullptr)
{
auto* btn = new juce::TextButton{ "REMOVE" };
btn->setComponentID(std::to_string(rowNumber));
btn->addListener(this);
existingComponentToUpdate = btn;
}
}return existingComponentToUpdate;
}
}

```

```

//Button Handling ** Own Codes **
void PlaylistComponent::buttonClicked(juce::Button* button)
{
    if (button == &importButton)
    {
        DBG("Import button clicked");
        importTracks();
        library.updateContent();
    }
    else if (button == &addToDeck1Button)
    {
        DBG("Add to Deck 1 clicked");
        loadInPlayer(deckGUI1);
    }
    else if (button == &addToDeck2Button)
    {
        DBG("Add to Deck 2 clicked");
        loadInPlayer(deckGUI2);
    }
    else
    {
        int id = std::stoi(button-
>getComponentID().toStdString());
        DBG(tracks[id].title + " removed from Library");
        deleteFromTracks(id);
        library.updateContent();
    }
}

// Track Management ** Own Codes **
void PlaylistComponent::loadInPlayer(DeckGUI* deckGUI)
{
    int selectedRow{ library.getSelectedRow() };
    if (selectedRow != -1)
    {
        DBG("Adding: " << tracks[selectedRow].title << " to
Player");
        deckGUI->loadFile(tracks[selectedRow].url);
    }
    else
    {
        juce::AlertWindow::showMessageBoxAsync(
juce::AlertWindow::InfoIcon,
"Add on to Deck Info:",
"Select track to add to deck",
"OK",

```

```

nullptr
);
}
}

void PlaylistComponent::importTracks(){
    DBG("Loading tracks from file...");
    //initialize file chooser
    auto fileChooserFlags =
    juce::FileBrowserComponent::canSelectFiles |
    juce::FileBrowserComponent::openMode |
    juce::FileBrowserComponent::canSelectMultipleItems;
    fChooser.launchAsync(fileChooserFlags, [this](const
    juce::FileChooser& fc)
    {
        auto files = fc.getResults(); // Get selected files
        for (const auto& file : files)
        {
            juce::String fileNameWithoutExtension =
            file.getFileNameWithoutExtension();
            if (!isInTracks(fileNameWithoutExtension)) // If not
            already loaded
            {
                loaded",
                Track newTrack{ file };
                juce::URL audioURL{ file };
                newTrack.length = getLength(audioURL);
                tracks.push_back(newTrack);
                DBG("loaded file: " << newTrack.title);
                library.updateContent();
            }
            else // Display info message
            {
                juce::AlertWindow::showMessageBoxAsync(
                juce::AlertWindow::InfoIcon,
                "Load information:",
                fileNameWithoutExtension + " already
                "OK"
            );
        }
    }
);

void PlaylistComponent::deleteFromTracks(int id)
tracks.erase(tracks.begin() + id);
}

```

```

{
}
{
}

bool PlaylistComponent::isInTracks(juce::String
fileNameWithoutExtension)
return (std::find(tracks.begin(), tracks.end(),
fileNameWithoutExtension) != tracks.end()); // Track Length Conversion ** Own Codes **

juce::String PlaylistComponent::getLength(juce::URL audioURL)
{
playerForParsingMetaData->loadFile(audioURL);
return secondsToMinutes(playerForParsingMetaData-
>getLengthInSeconds());
}

{
juce::String PlaylistComponent::secondsToMinutes(double seconds)
int secondsRounded = static_cast<int>(std::round(seconds));
return juce::String(secondsRounded / 60) + ":" +
juce::String(secondsRounded % 60).paddedLeft('0', 2);
}

// Library Management ** Own Codes **

void PlaylistComponent::searchLibrary(juce::String searchText)
{
DBG("Searching library for: " << searchText);
if (searchText != "")
{
int rowNumber = whereInTracks(searchText);
library.selectRow(rowNumber);
}
else
{
}
library.deselectAllRows();
}

{
int PlaylistComponent::whereInTracks(juce::String searchText)
auto it = std::find_if(tracks.begin(), tracks.end(),
[&searchText](const Track& obj)
{ return obj.title.contains(searchText); });
return (it != tracks.end()) ? std::distance(tracks.begin(),
it) : -1;
}

{
void PlaylistComponent::saveLibrary()
// create .csv to save library

```

```

std::ofstream myLibrary("my-library.csv");
// save library to file
for (Track& t : tracks)
{
myLibrary << t.file.getFullPathName() << "," << t.length <<
"\n";
}
}

void PlaylistComponent::loadLibrary(){
std::ifstream myLibrary("my-library.csv");
std::string filePath, length;
while (getline(myLibrary, filePath, ','))
{
Track newTrack{ juce::File{ filePath } };
getline(myLibrary, length);
newTrack.length = length;
tracks.push_back(newTrack);
}
}

*****
*****

```

12. Track.h

```

*****
*****

/*
=====

=====

Track.h
Created: 13 Feb 2025 8:42:38pm
Author: Devon Xavior
** All codes wrote without assistance **
=====

=====

*/

#pragma once
#include <JuceHeader.h>
// Track Class
class Track
{
public:
// Constructor
Track(juce::File _file);
// Public member variables
juce::File file; // Audio file
juce::String title; // File name (without extension)

```

```
juce::URL url; // URL derived from the file
juce::String length; // Track duration in "mm:ss" format
// Operator Overloading
bool operator==(const juce::String& other) const;
};*****
*****
```

13. Track.cpp

```
*****
*****

/*
=====

=====

Track.cpp
Created: 13 Feb 2025 8:42:38pm
Author: Devon Xavior
** All codes wrote without assistance **
=====

=====

*/

#include "Track.h"
// Constructor
Track::Track(juce::File _file)
: file(_file),
title(_file.getFileNameWithoutExtension()),
url(juce::URL{ _file })
DBG("Created track with title: " << title);
{
}

// Operator Overloading
bool Track::operator==(const juce::String& other) const
{
return title == other;
}

*****
*****

END
*****
*****
```